

Of course, you might also want to remove logical volumes once in a while. The command to do that is dead simple:

```
$: sudo lvremove /dev/VolGroup/Swap
```

Replace the device node in the above command to match the volume you want to remove. Just remember to unmount the volume first.

Remember that unless you're using GRUB 2, your /boot partition cannot reside inside an LVM. If you're using EFI, you'd use the ESP as a boot partition, but if you're on BIOS systems and you don't like GRUB 2, you're going to need a separate standard partition to boot from, preferably formatted with ext2. I'll show you how to boot from LVM using GRUB 2 later in the article.

That's it about creating LVM sets. Now let's move on to growing, shrinking and throwing things around.

Extending LVM sets

Let's assume we want 20 additional gigabytes on the 'Root' volume we created. The command we'd need to run is as follows:

```
$: sudo lvresize -L +20G VolGroup/Root
```

Yes, that's just *VolGroup/Root*. You don't have to supply the device node. Anyway, the logical volume has just grown, but we still need to grow the filesystem inside it to make use of the additional space available. Assuming you're running ext4 (LVM and ext4 were just made for each other it seems), you can use the following command:

```
$: sudo resize2fs /dev/VolGroup/Root
```

And the filesystem will grow to take up all the space in the volume. What if you needed to reduce the size of your volume? That's easy too, though not as easy as growing one. We'll do it in three steps.

First, you need to figure out the new, reduced size of the volume. Let's say you want to reduce 'Root' to 48 GB. Let's begin by resizing the filesystem inside 'Root' to take up 47 GB of space. Let's make it smaller than we need to so that when we resize the logical volume, we don't accidentally slice off the filesystem's tail.

```
$: sudo resize2fs /dev/VolGroup/Root 47G
```

Of course, this won't work if you have more than 47 GB of data in 'Root'. Anyway, once we've got the filesystem sorted, we'll actually reduce the LV's size, as follows:

```
$: sudo lvreduce -L 48G VolGroup/Root
```

And that's done. Now, to grow the filesystem inside to take up the additional 1 GB or so that we left as an error margin, just run the following:

```
$: sudo resize2fs /dev/VolGroup/Root
```

So we've seen two commands that deal with the sizing of logical volumes—*lvresize* and *lvreduce*. There's a third, called *lvextend*.

The basic idea is that you use *lvreduce* and *lvextend* when you know the exact final size of the volume you want. If you want to grow 'Root' to 100 GB, you can use the command given below:

```
$: sudo lvextend -L 100G VolGroup/Root
```

Similarly, to shrink it to 32 GB, you can run the following:

```
$: sudo lvreduce -L 32G VolGroup/Root
```

And as a shortcut, if you want your volume to grow to the maximum size possible, taking all the free space in the volume group, run the command given below:

```
$: sudo lvextend -l +100%FREE VolGroup/Root
```

Yes, that last command uses a lowercase 'l'. So why would you need *lvresize*? Well, for relative resizing. Say you want to grow 'Root' by 4 GB, you can run the following command:

```
$: sudo lvresize -L +4G VolGroup/Root
```

Or if you want to shrink it by 2 GB, you run:

```
$: sudo lvresize -L -2G VolGroup/Root
```

A bit convoluted, I know, but still pretty nice, right? Just remember to resize the filesystems that are inside, afterwards.

LVM and more disks

So far, I've assumed you have some free space in your volume group. What if you've run out? What if you need to bring in some new hardware?

Let's suppose you've just installed a new hard drive into your system and want to dedicate the whole thing to LVM. Let's say the drive shows up as */dev/sdb*.

Start off by creating a partition inside the hard drive that takes up the whole drive, and give it a type ID of 8e00. It doesn't matter if you use GPT or MBR. You could even use Apple Partition Map. All that is needed is a valid partition table on the drive. (Strictly speaking, you don't need it. In fact, you could very well just use */dev/sdb* as a superfloppy